

Reviewer Pack — Minimal Rank of Tropicalized Graph Automorphism Groups vs AC...

Entry de66c8f2cb7e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 00:27:26 UTC

Minimal Rank of Tropicalized Graph Automorphism Groups vs AC⁰ Circuit Size

Entry ID: de66c8f2cb7e **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-25 00:27:26 UTC

1. Conjecture

Field A (mathematical branch): Tropical Graph Theory (Automorphism Groups) **Field B** (complexity object): Complexity Theory: AC⁰ Circuit Complexity

Statement:

[‘For any given graph G , the minimal rank of the tropicalized automorphism group of G , denoted as $T_rank(G)$, is upper bounded by the size of its smallest AC⁰ circuit. Specifically, for all graphs G with n vertices, $T_rank(G) \leq 2^n - O(n)$.’, ‘Equivalently, if there exists a graph G such that $T_rank(G) > 2^n - O(n)$, then G cannot be computed by an AC⁰ circuit of size less than 2^n .’, ‘Furthermore, for any graph G with at least n vertices, there exists a tropicalized automorphism group representation whose minimal rank is exactly $2^n - O(n)$.’]

Rationale (proposer’s reasoning):

[‘Tropical graph theory provides a way to study the symmetries of graphs in a more algebraic setting. The automorphism groups of graphs capture these symmetries, and their tropicalizations may reveal deep connections with circuit complexity.’, ‘If the minimal rank of the tropicalized automorphism group is closely related to the size of an AC⁰ circuit for the graph, it could indicate that certain symmetries are computationally fundamental in understanding the graph’s structure.’, ‘This connection has not been thoroughly explored and offers a novel avenue for understanding the interplay between algebraic graph theory and complexity theory.’]

Taxonomy category: TROPICAL_AUTOMORPHISM_GROUP (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c199f88a91851f43

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if $T_rank(G) \leq (2^n - O(n))$ for all graphs G with n vertices, where $T_rank(G)$ is the minimal rank of the tropicalized automorphism group and $O(n)$

represents a term that grows no faster than linearly with n . The conjecture is falsified if there exists a graph G such that $T_{\text{rank}}(G) > 2^n - O(n)$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'tropical graph theory' AND 'automorphism groups' AND 'AC⁰ circuit complexity' - 'minimal rank' OF 'tropicalized graph automorphism groups' AND 'upper bound' ON 'AC⁰ circuit size' - 'tropicalization' IN 'graph theory' AND 'circuit complexity' = 'AC⁰'

Top relevant hits considered: - [<http://arxiv.org/abs/1207.2443v2>] Tropical Teichmuller and Siegel spaces - [<http://arxiv.org/abs/1204.6154v2>] Local Tropicalization - [<http://arxiv.org/abs/1204.3875v2>] Tropicalizing vs Compactifying the Torelli morphism - [<http://arxiv.org/abs/1509.06670v2>] Automorphism Groups and Invariant Theory on PN - [<http://arxiv.org/abs/math/0609425v1>] Upper Bounds on the Automorphism Group of a Graph - [<http://arxiv.org/abs/1109.2952v4>] Upper bound on distance in the pants complex - [<http://arxiv.org/abs/0711.2360v2>] Every longest circuit of a 3-connected, $K_{3,3}$ -minor free graph has a chord - [<http://arxiv.org/abs/1402.2589v3>] Partitioning Perfect Graphs into Stars

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_graph(n):
        edges = set()
        for i in range(n):
            for j in range(i + 1, n):
                if random.choice([True, False]):
                    edges.add((i, j))
        return {i: list(j) for i, j in enumerate(edges)}

    def tropicalized_automorphism_group(G):
        # Placeholder function to compute T_rank(G)
```

```

    # This is a stub and should be replaced with actual computation
    n = len(G)
    return 2**n - 1

def ac0_circuit_size(G):
    # Placeholder function to estimate the size of the smallest AC0 circuit for G
    # This is a stub and should be replaced with actual computation
    n = len(G)
    return 2**n

n = random.randint(5, 40) # Sweep n through at least 4 distinct sizes inside each trial
G = generate_random_graph(n)
T_rank_G = tropicalized_automorphism_group(G)
ac0_size = ac0_circuit_size(G)

return {
    "metric_name": "T_rank(G)",
    "metric_value": T_rank_G,
    "instances_tested": 1,
    "conjecture_holds": T_rank_G <= ac0_size - n,
    "counterexample": "" if T_rank_G <= ac0_size - n else f"Graph with {n} vertices has T_rank
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] if len(sys.argv) > 1 else [random.getrandbits(32) for _

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fra
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fra
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        counterexample = next(r["counterexample"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

es has T_rank(G) = 4611686018427387903, but AC0 circuit size is at least 4611686018427387904'}
TRIAL: {'metric_name': 'T_rank(G)', 'metric_value': 29230032746618058364073696654325660393118650859
TRIAL: {'metric_name': 'T_rank(G)', 'metric_value': 268435455, 'instances_tested': 1, 'conjecture_ho
TRIAL: {'metric_name': 'T_rank(G)', 'metric_value': 5192296858534827628530496329220095, 'instances_
TRIAL: {'metric_name': 'T_rank(G)', 'metric_value': 19615942923083377386986841947523957550319860763

```


Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/de66c8f2cb7e.tar.gz (if generated)